3. PROGRAMACIÓN DINÁMICA
Ejemplo: Cambio de Monedas

- Cambio de monedas:

  Dada una cantidad P, hay que devolver esa cantidad usando el menor número posible de monedas. Hay **monedas ilimitadas** de valores Vi.

3. PROGRAMACIÓN DINÁMICA
Ejemplo: Cambio de Monedas

- Reformulación del problema:
  - Devolver una cantidad entera ***M*** en el menor número posible de monedas. Se cuenta con una cantidad infinita de monedas de valores distintos v_1, v_2, …, v_n.

- Estrategia recursiva:
  - El cambio mínimo *M* con un conjunto de monedas $v_1 \ldots v_n$ será:

$$\text{Cambio}(\{v_1 \ldots v_n\}, M) = \min \begin{cases} \text{Cambio}(\{v_1 \ldots v_{n-1}\}, C) \\ 1 + \text{Cambio}(\{v_1 \ldots v_n\}, C - v_n) \end{cases}$$

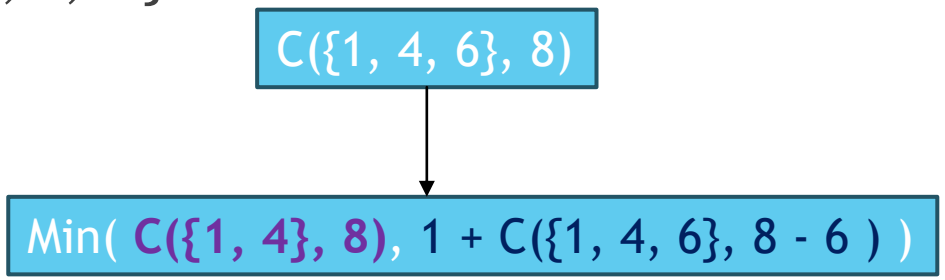
3. PROGRAMACIÓN DINÁMICA
Ejemplo: Cambio de Monedas

V = { 1, 4, 6 } M = 8

C({1, 4, 6}, 8)

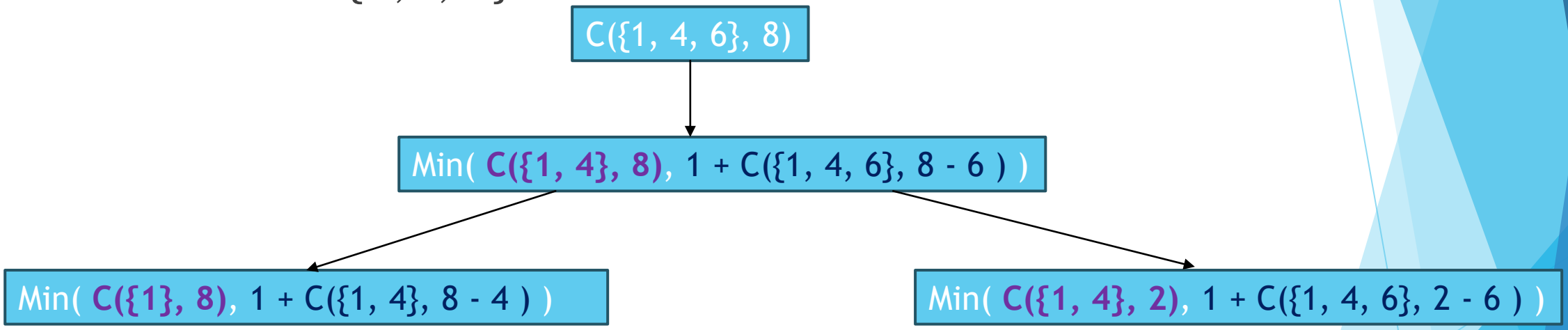
3. PROGRAMACIÓN DINÁMICA
Ejemplo: Cambio de Monedas

V = { 1, 4, 6 } M = 8



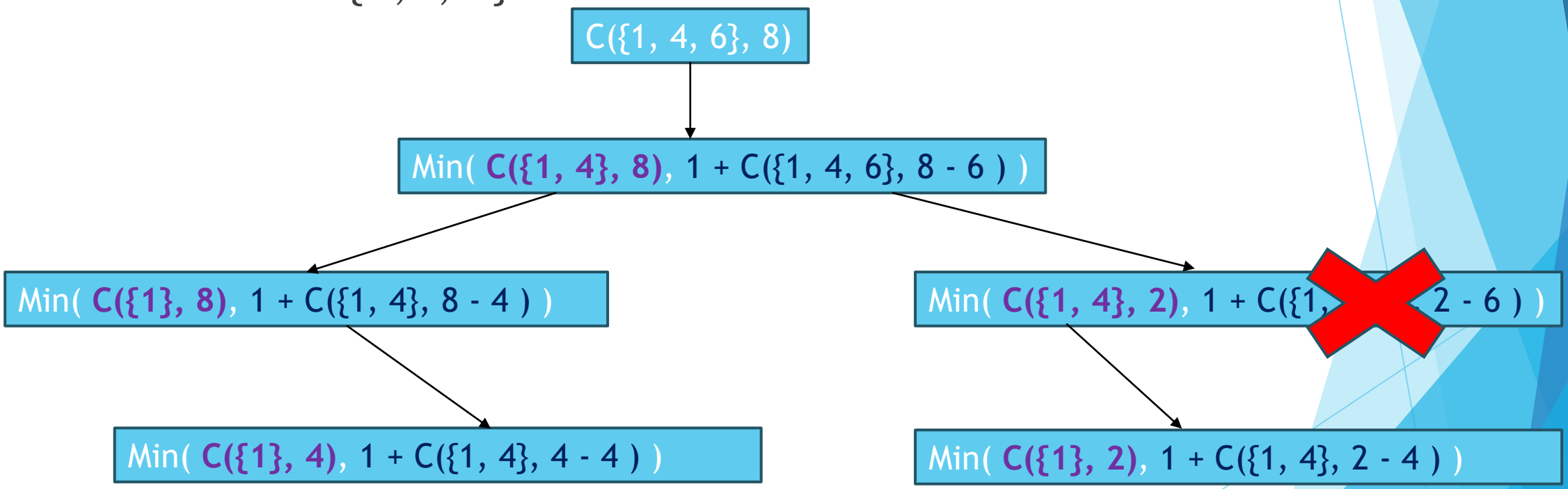
3. PROGRAMACIÓN DINÁMICA
Ejemplo: Cambio de Monedas

V = { 1, 4, 6 } M = 8



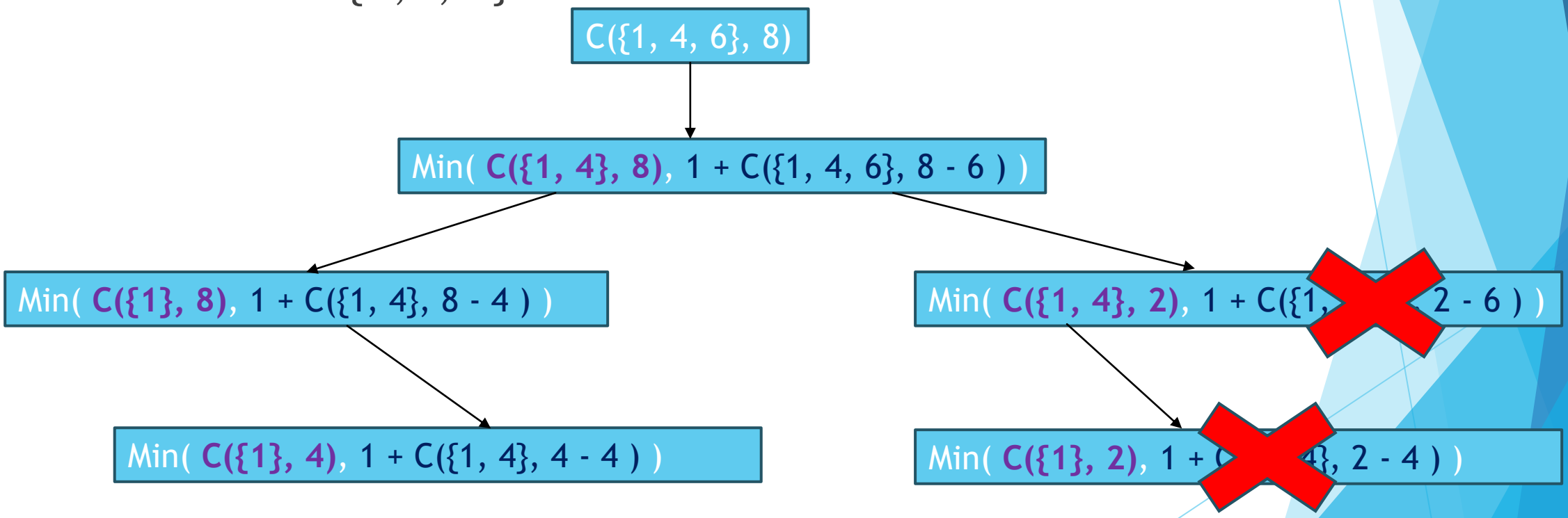
3. PROGRAMACIÓN DINÁMICA
Ejemplo: Cambio de Monedas

V = { 1, 4, 6 } M = 8



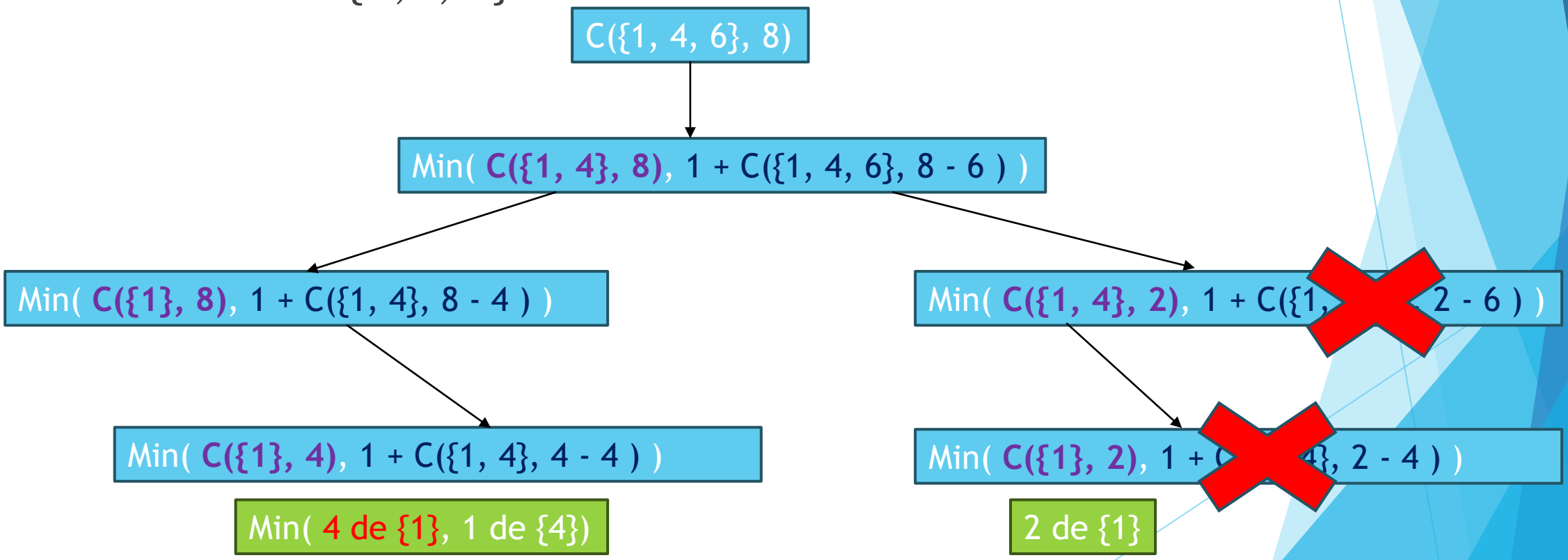
3. PROGRAMACIÓN DINÁMICA
Ejemplo: Cambio de Monedas

V = { 1, 4, 6 } M = 8

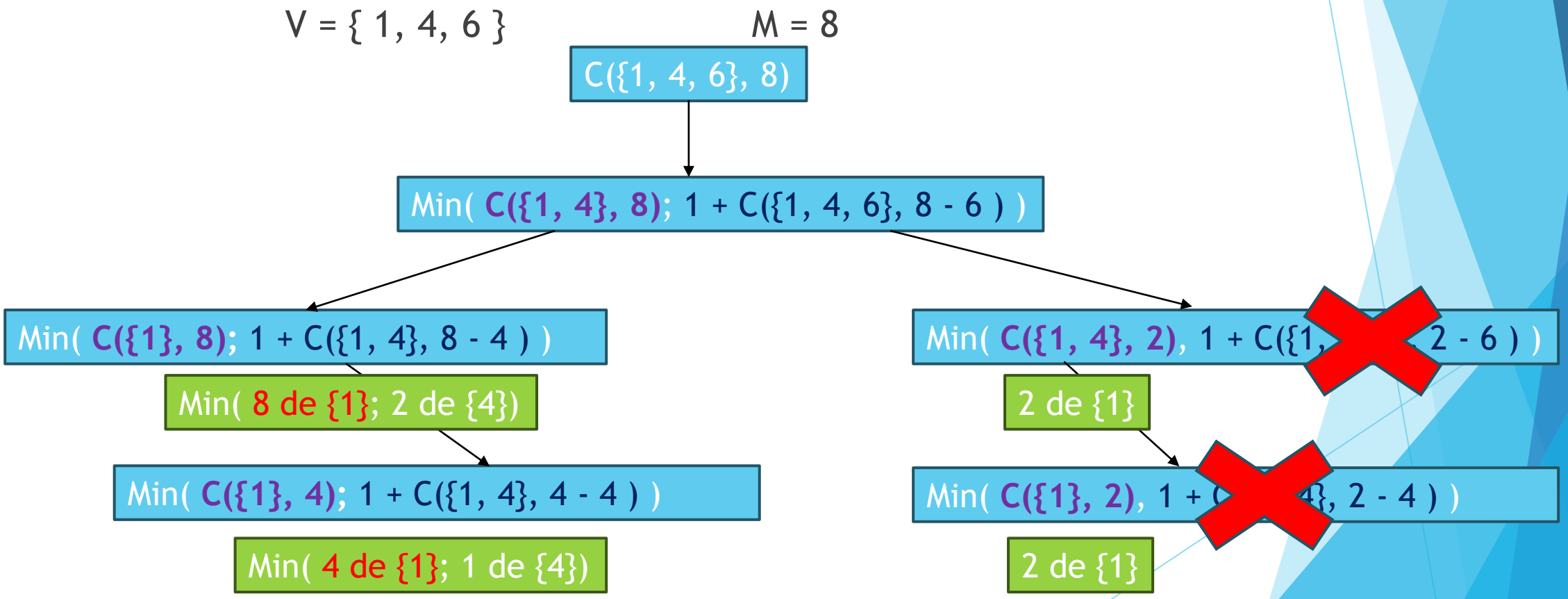


3. PROGRAMACIÓN DINÁMICA
Ejemplo: Cambio de Monedas

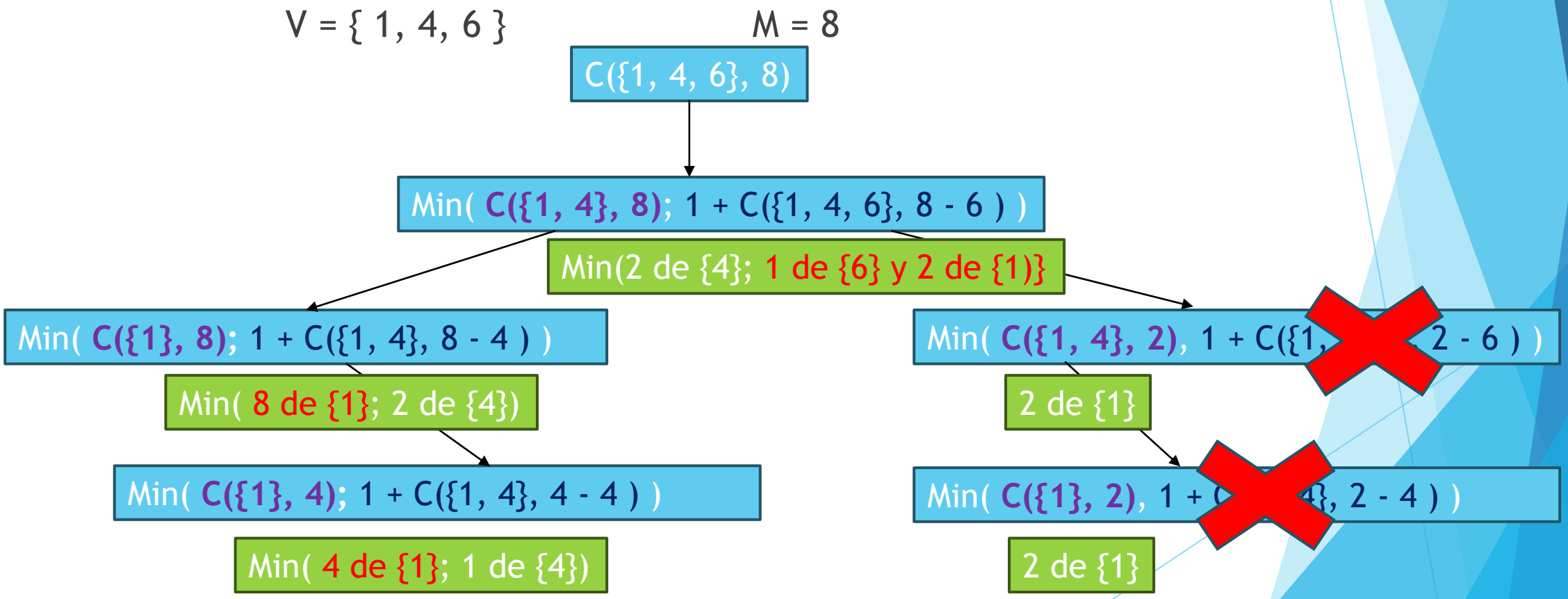
V = { 1, 4, 6 } M = 8



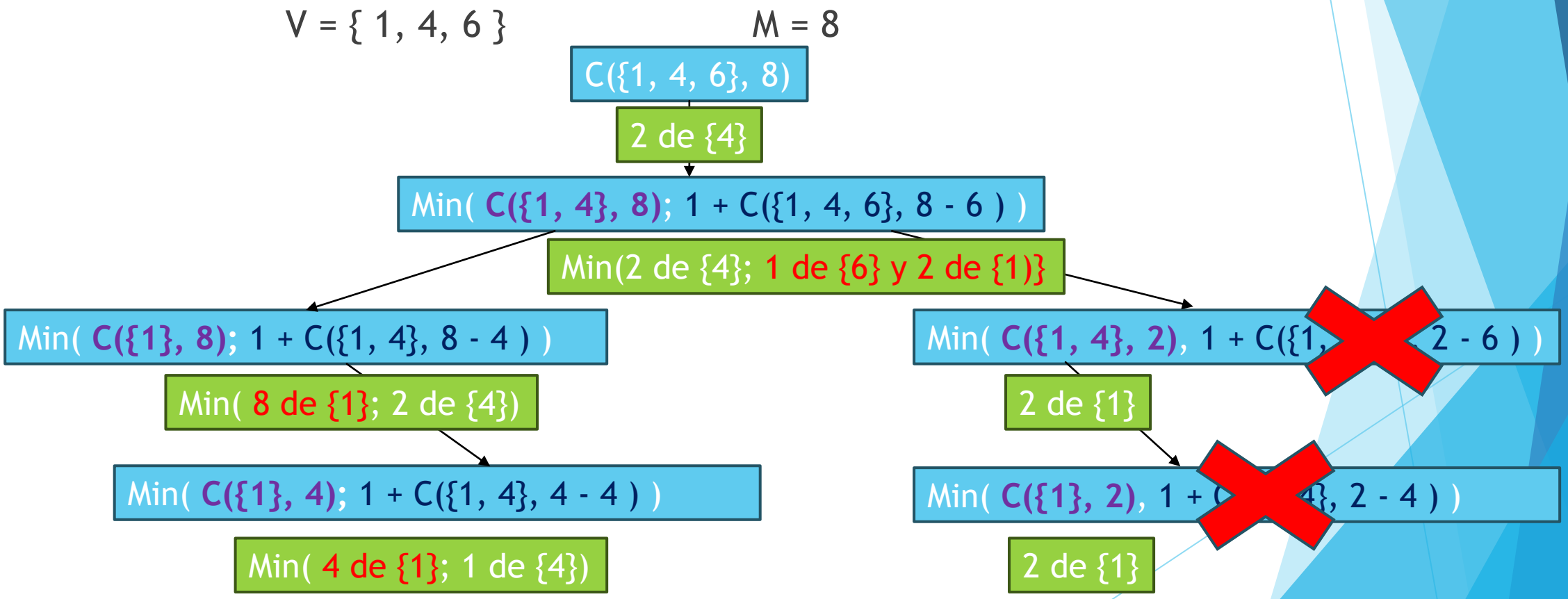
3. PROGRAMACIÓN DINÁMICA
Ejemplo: Cambio de Monedas



3. PROGRAMACIÓN DINÁMICA
Ejemplo: Cambio de Monedas



3. PROGRAMACIÓN DINÁMICA
Ejemplo: Cambio de Monedas



3. PROGRAMACIÓN DINÁMICA
Ejemplo: Cambio de Monedas

- Reformulación iterativa:
  - Construcción de tabla T de tamaño n x (M + 1), donde almacenaremos el mínimo de monedas de cada tipo para resolver cada subproblema (i, m).
  - El índice i indica la moneda Vi y m la cantidad a devolver.
  - Inicializada a None excepto para m = 0 que será 0.

	0	1	2	3	4	5	6	7	8
0 *{1}*	0	None	None	None	None	None	None	None	None
1 *{4}*	0	None	None	None	None	None	None	None	None
2 *{6}*	0	None	None	None	None	None	None	None	None

3. PROGRAMACIÓN DINÁMICA
Ejemplo: Cambio de Monedas

- Cálculo de la primera fila:

	0	1	2	3	4	5	6	7	8
0 *{1}*	0	1	2	3	4	5	6	7	8
1 *{4}*	0	None	None	None	None	None	None	None	None
2 *{6}*	0	None	None	None	None	None	None	None	None

3. PROGRAMACIÓN DINÁMICA
Ejemplo: Cambio de Monedas

- Cálculo del resto de filas:

i = 1 →

	0	1	2	3	4	5	6	7	8
0 *{1}*	0	1	2	3	4	5	6	7	8
1 *{4}*	**0**	**1**	**2**	**3**	None	None	None	None	None
2 *{6}*	0	None	None	None	None	None	None	None	None

3. PROGRAMACIÓN DINÁMICA
Ejemplo: Cambio de Monedas

- Cálculo del resto de filas:

i = 1 →

	0	1	2	3	4	5	6	7	8
0 {1}	0	1	2	3	4 ?	5	6	7	8
1 {4}	0 ?	1	2	3	1	None	None	None	None
2 {6}	0	None	None	None	None	None	None	None	None

3. PROGRAMACIÓN DINÁMICA
Ejemplo: Cambio de Monedas

- Cálculo del resto de filas:

i = 1 →

	0	1	2	3	4	5	6	7	8
0 {1}	0	1	2	3	4	5 ?	6	7	8
1 {4}	0	1 ?	2	3	1	2	None	None	None
2 {6}	0	None	None	None	None	None	None	None	None

3. PROGRAMACIÓN DINÁMICA
Ejemplo: Cambio de Monedas

- Cálculo del resto de filas:

i = 1 →

	0	1	2	3	4	5	6	7	8
0 {1}	0	1	2	3	4	5	6 ?	7	8
1 {4}	0	1	2 ?	3	1	2	3	None	None
2 {6}	0	None	None	None	None	None	None	None	None

3. PROGRAMACIÓN DINÁMICA
Ejemplo: Cambio de Monedas

- Cálculo del resto de filas:

i = 1 →

	0	1	2	3	4	5	6	7	8
0 {1}	0	1	2	3	4	5	6	7 ?	8
1 {4}	0	1	2	3 ?	1	2	3	4	None
2 {6}	0	None	None	None	None	None	None	None	None

3. PROGRAMACIÓN DINÁMICA
Ejemplo: Cambio de Monedas

- Cálculo del resto de filas:

i = 1 →

	0	1	2	3	4	5	6	7	8
0 {1}	0	1	2	3	4	5	6	7	8 ?
1 {4}	0	1	2	3	1 ?	2	3	4	2
2 {6}	0	None	None	None	None	None	None	None	None

3. PROGRAMACIÓN DINÁMICA
Ejemplo: Cambio de Monedas

- Cálculo del resto de filas:

i = 2 →

	0	1	2	3	4	5	6	7	8
0 *{1}*	0	1	2	3	4	5	6	7	8
1 *{4}*	0	1	2	3	1	2	3	4	2
2 *{6}*	**0**	**1**	**2**	**3**	**1**	**2**	None	None	None

3. PROGRAMACIÓN DINÁMICA
Ejemplo: Cambio de Monedas

- Cálculo del resto de filas:

i = 2 →

	0	1	2	3	4	5	6	7	8
0 *{1}*	0	1	2	3	4	5	6	7	8
1 *{4}*	0	1	2	3	1	2	3 ?	4	2
2 *{6}*	0 ?	1	2	3	1	2	1	None	None

3. PROGRAMACIÓN DINÁMICA
Ejemplo: Cambio de Monedas

- Cálculo del resto de filas:

i = 2 →

	0	1	2	3	4	5	6	7	8
0 *{1}*	0	1	2	3	4	5	6	7	8
1 *{4}*	0	1	2	3	1	2	3	4 ?	2
2 *{6}*	0	1 ?	2	3	1	2	1	**2**	None

3. PROGRAMACIÓN DINÁMICA
Ejemplo: Cambio de Monedas

- Cálculo del resto de filas:

i = 2 →

	0	1	2	3	4	5	6	7	8
0 {1}	0	1	2	3	4	5	6	7	8
1 {4}	0	1	2	3	1	2	3	4	2 ?
2 {6}	0	1	2 ?	3	1	2	1	2	2

3. PROGRAMACIÓN DINÁMICA
Ejemplo: Cambio de Monedas

Devolver T[n – 1, M]

	0	1	2	3	4	5	6	7	8
0 *{1}*	0	1	2	3	4	5	6	7	8
1 *{4}*	0	1	2	3	1	2	3	4	2
2 *{6}*	0	1	2	3	1	2	1	2	2

El mínimo de monedas a devolver es 2.

3. PROGRAMACIÓN DINÁMICA
Ejemplo: Cambio de Monedas

- Recuperar la solución:

	0	1	2	3	4	5	6	7	8
0 *{1}*	0	1	2	3	4	5	6	7	8
1 *{4}*	0	1	2	3	1	2	3	4	2
2 *{6}*	0	1	2	3	1	2	1	2	2

3. PROGRAMACIÓN DINÁMICA
Ejemplo: Cambio de Monedas

- Recuperar la solución:

j

	0	1	2	3	4	5	6	7	8
0 *{1}*	0	1	2	3	4	5	6	7	8
1 *{4}*	0	1	2	3	1	2	3	4	2
2 *{6}*	0	1	2	3	1	2	1	2	2

i

Sol:

0 {1}	1 {4}	2 {6}
0	0	0

3. PROGRAMACIÓN DINÁMICA
Ejemplo: Cambio de Monedas

- Recuperar la solución:

j

	0	1	2	3	4	5	6	7	8
0 *{1}*	0	1	2	3	4	5	6	7	8
1 *{4}*	0	1	2	3	1	2	3	4	2
2 *{6}*	0	1	2	3	1	2	1	2	2

i

Sol:

0 {1}	1 {4}	2 {6}
0	0	0

3. PROGRAMACIÓN DINÁMICA
Ejemplo: Cambio de Monedas

- Recuperar la solución:

i ⇒ (fila) j ⇓ (columna)

	0	1	2	3	4	5	6	7	8
0 *{1}*	0	1	2	3	4	5	6	7	8
1 *{4}*	0	1	2	3	1	2	3	4	2
2 *{6}*	0	1	2	3	1	2	1	2	2

Sol:

0 {1}	1 {4}	2 {6}
0	0	0

3. PROGRAMACIÓN DINÁMICA
Ejemplo: Cambio de Monedas

- Recuperar la solución:

i →

	0	1	2	3	4	5	6	7	8
0 {1}	0	1	2	3	4	5	6	7	8
1 {4}	0	1	2	3	1	2	3	4	2
2 {6}	0	1	2	3	1	2	1	2	2

j ↓

Sol:

0 {1}	1 {4}	2 {6}
0	1	0

3. PROGRAMACIÓN DINÁMICA
Ejemplo: Cambio de Monedas

- Recuperar la solución:



	0	1	2	3	4	5	6	7	8
0 {1}	0	1	2	3	4	5	6	7	8
1 {4}	0	1	2	3	1	2	3	4	2
2 {6}	0	1	2	3	1	2	1	2	2

Sol:

0 {1}	1 {4}	2 {6}
0	1	0

3. PROGRAMACIÓN DINÁMICA
Ejemplo: Cambio de Monedas

- Recuperar la solución:



	0	1	2	3	4	5	6	7	8
0 {1}	0	1	2	3	4	5	6	7	8
1 {4}	0	1	2	3	1	2	3	4	2
2 {6}	0	1	2	3	1	2	1	2	2

Sol:

0 {1}	1 {4}	2 {6}
0	1	0

3. PROGRAMACIÓN DINÁMICA
Ejemplo: Cambio de Monedas

- Recuperar la solución:





	0	1	2	3	4	5	6	7	8
0 {1}	0	1	2	3	4	5	6	7	8
1 {4}	0	1	2	3	1	2	3	4	2
2 {6}	0	1	2	3	1	2	1	2	2

Sol:

0 {1}	1 {4}	2 {6}
0	2	0

3. PROGRAMACIÓN DINÁMICA
Ejemplo: Cambio de Monedas

- Recuperar la solución:

j

i

	0	1	2	3	4	5	6	7	8
0 {1}	0	1	2	3	4	5	6	7	8
1 {4}	0	1	2	3	1	2	3	4	2
2 {6}	0	1	2	3	1	2	1	2	2

Sol:

0 {1}	1 {4}	2 {6}
0	2	0

3. PROGRAMACIÓN DINÁMICA
Ejemplo: Cambio de Monedas

- Recuperar la solución:

j

i

	0	1	2	3	4	5	6	7	8
0 {1}	0	1	2	3	4	5	6	7	8
1 {4}	0	1	2	3	1	2	3	4	2
2 {6}	0	1	2	3	1	2	1	2	2

Sol:

0 {1}	1 {4}	2 {6}
0	2	0

3. PROGRAMACIÓN DINÁMICA
Ejemplo: Cambio de Monedas

- Recuperar la solución:

Devolverá DOS monedas de 4 u.v.

0 {1}	1 {4}	2 {6}
0	2	0

3. PROGRAMACIÓN DINÁMICA
Ejemplo: Cambio de Monedas

```
C( V = {monedas}, int M):
    T[..., ...] = None
    T[0, 0] = 0
    Para j en Rango(1, M + 1)
        Si (j >= V[0] Y T[0, j - V[0]] != None)
            T[0, j] = 1 + T[0, j - V[0]]
    Para i en Rango(1, n)
        T[i, 0] = 0
        Para j en Rango(1, M + 1)
            Si (j < V[i] O T[i, j - V[i]] == None)
                T[i, j] = V[i - 1, j]
            SiNo Si (T[i - 1, j] != None)
                T[i, j] = min(T[i - 1, j], 1 + T[i, j - V[i])
            SiNo
                T[i, j] = 1 + T[i, j - V[i]]
    Devolver T[n - 1, M]
```

Cálculo primera fila

Cálculo del resto de la tabla

3. PROGRAMACIÓN DINÁMICA
Ejemplo: Cambio de Monedas

- Recuperar la solución:

```
Sol = {0 . . . 0}
i = n - 1
j = M
Mientras (i != 0 Y j !=0)
        Si i == 0 O T[i, j] != T[i - 1, j]
                Sol[i] = Sol[i] + 1
                j = j - V[i]
        SiNo
                i = i - 1
Devolver Sol
```

- Representación valor “infinito”:

```
#include<limits>
int inf_val = std::numeric_limits<int>::max();
```

- Mínimo de dos valores:

```
#include<algorithm>
std::min(a, b);
```

- Matriz f x c con std::vectors:

```
std::vector<std::vector<int>> m(f, std::vector<int>(c, 0));
```

- Implementar función monedas:
  - Entradas: -Vector **m** de enteros con los valores de cada moneda.
    -Valor entero **d** con el valor a devolver.
  - Salida: -Vector **s** de enteros con las cantidades a devolver de cada moneda, en los parámetros de la función.

- Caso de la mochila:
  - Capacidad (peso) máximo **M**.
  - Elementos con pesos **p** y beneficios **b**.
  - Llenar la mochila de manera que se maximice el beneficio.

$$g(i,C) \begin{cases} g(i-1,C) & ,c_i > C \\ max(g(i-1,C), g(i-1,C-c_i)+b_i) & ,c_i <= C \end{cases}$$

siendo $g(i,C)$ la máxima ganancia posible al considerar sólo los primeros i objetos y capacidad C, siendo $0 \leq i \leq N$, $0 \leq C \leq M$, con el objetivo final de calcular $g(N,M)$.

- **M** = 15, **p** = (9, 6, 5), **b** = (38, 40, 24)

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	0															
2	0															
3	0															

- **M** = 15, **p** = (9, 6, 5), **b** = (38, 40, 24)

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	0	38	38	38	38	38	38	38
2	0															
3	0															

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	0	38	38	38	38	38	38	38
2	0	0	0	0	0	0	40	40	40	40	40	40	40	40	40	78
3	0															

- **M** = 15, **p** = (9, 6, 5), **b** = (38, 40, 24)

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	0	38	38	38	38	38	38	38
2	0	0	0	0	0	0	40	40	40	40	40	40	40	40	40	78
3	0	0	0	0	0	24	40	40	40	40	40	64	64	64	64	78

- Solución óptima: elegir los elementos 0 y 1 (1 y 2 en la tabla) para un beneficio de 78.

- M = 5, p = (1, 3, 2, 4), b = (10, 20, 15, 20) -> elegir los elementos 1 y 2 para un beneficio de 35.

- Implementar función mochila:
  - Entradas: -Vector **p** de enteros con los pesos de cada elemento.
    -Vector **b** de enteros con el beneficio de cada elemento.
    -Valor entero **m** con el peso máximo.
  - Salida: -Vector **s** de enteros indicando si cada elemento ha sido elegido (1) o no (0), en los parámetros de la función.
    -Beneficio máximo.